

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования
**«Белорусский государственный университет
информатики и радиоэлектроники»**

Факультет компьютерных систем и сетей
Кафедра программного обеспечения информационных технологий

ОТЧЁТ

по лабораторной работе №3

по дисциплине «Стили и методы программирования»

**Тема: «Введение в функциональное программирование.
Основы разработки на языке функционального
программирования»**

Выполнил:

Лебедев Артем Владимирович
магистрант факультета
компьютерных систем и сетей
группы 556301

Преподаватель:

Парамонов Антон Иванович
кандидат технических наук,
доцент, заведующий кафедрой
информационных систем и техноло-
гий ИТТ БГУИР

Минск, 2026

Содержание

Введение	3
1 ОПИСАНИЕ ИСПОЛЬЗУЕМОГО ИНТЕРПРЕТАТОРА И СРЕДЫ РАЗРАБОТКИ	4
2 СОЗДАНИЕ ПРОЕКТА И ЗАПУСК ПРОГРАММЫ.....	5
3 ФРАГМЕНТЫ КОДА ИНДИВИДУАЛЬНЫХ ЗАДАНИЙ	6
3.1 Задание 2.1 – вычисление математического выражения	6
3.2 Задание 2.2 – функция $Y(x)$ с условным предложением COND	7
3.3 Задание 2.3 – рекурсивная функция вычисления факториала	8
3.4 Задание 3 – обработка текста с использованием функционалов	9
4 РЕАЛИЗАЦИЯ ЗАДАНИЙ НА ИМПЕРАТИВНОМ ЯЗЫКЕ PYTHON	10
4.1 Задание 2.1 – вычисление математического выражения	10
4.2 Задание 2.2 – функция $Y(x)$	10
4.3 Задание 2.3 – вычисление факториала	10
4.4 Задание 3 – обработка текста.....	11
5 ОСНОВНЫЕ КОНСТРУКЦИИ ЯЗЫКА	12
6 СРАВНЕНИЕ ФУНКЦИОНАЛЬНОГО И ИМПЕРАТИВНОГО ПОДХОДОВ	13
7 РЕЗУЛЬТАТЫ РАБОТЫ КОДА	14
Заключение	16
Список использованных источников	17

ВВЕДЕНИЕ

Функциональное программирование – парадигма, в которой вычисления рассматриваются как применение математических функций. Ключевыми принципами являются: отсутствие побочных эффектов, неизменяемость данных, использование функций высшего порядка и рекурсии вместо циклов.

Язык Common Lisp является одним из старейших и наиболее влиятельных функциональных языков программирования, который по-прежнему активно используется в академической среде и промышленных системах. Lisp послужил основой для развития многих современных функциональных языков.

Целью данной лабораторной работы является изучение основ функционального программирования на примере языка Common Lisp: реализация математических выражений, условных конструкций, рекурсивных функций, а также обработка текста с использованием функционалов.

1 ОПИСАНИЕ ИСПОЛЬЗУЕМОГО ИНТЕРПРЕТАТОРА И СРЕДЫ РАЗРАБОТКИ

Для выполнения заданий выбран язык Common Lisp с интерпретатором SBCL (Steel Bank Common Lisp).

В качестве альтернативной среды разработки использован Visual Studio Code с бесплатным расширением Alive (Common Lisp). Расширение устанавливается из Visual Studio Marketplace (<https://marketplace.visualstudio.com/items?itemName=rheller.alive>) и превращает редактор в интерактивную среду для написания и выполнения Lisp-кода с поддержкой REPL, подсветки синтаксиса и пошагового выполнения выражений.

Характеристики среды выполнения:

- интерпретатор: SBCL (Steel Bank Common Lisp) версии 2.6.5;
- дополнительный язык для сравнения: Python 3;
- платформа: macOS (darwin, arm64);
- редактор: Visual Studio Code 1.122 с расширением Alive (rheller.alive);
- дополнительный способ запуска: терминал (`sbcl -script имя_файла.lisp`).

SBCL является высокопроизводительной реализацией стандарта ANSI Common Lisp. Он включает компилятор в машинный код, что обеспечивает высокую скорость выполнения, а также интерактивный REPL для пошаговой разработки и отладки.

Установка на macOS:

- SBCL: `brew install sbcl`;
- Visual Studio Code: `brew install -cask visual-studio-code`;
- расширение Alive: `code -install-extension rheller.alive`.

2 СОЗДАНИЕ ПРОЕКТА И ЗАПУСК ПРОГРАММЫ

Для создания и запуска программы на Common Lisp выполняются следующие действия:

1 Создать текстовый файл с расширением `.lisp` в текстовом редакторе.

2 Написать код программы, сохранить файл.

3 Запустить программу из терминала командой:

```
sbcl -script имя_файла.lisp
```

Альтернативные способы запуска:

- интерактивный режим REPL: запустить `sbcl`, затем загрузить файл командой `(load "имя_файла.lisp")`;
- через VS Code с расширением Alive: открыть папку проекта, запустить REPL (команда «Start REPL»), выполнять код кнопкой «Evaluate» или через контекстное меню;
- из терминала: `sbcl -script имя_файла.lisp`.

3 ФРАГМЕНТЫ КОДА ИНДИВИДУАЛЬНЫХ ЗАДАНИЙ

3.1 Задание 2.1 – вычисление математического выражения

Вариант 1. Вычислить:

$$\sin\left(\frac{mn+2}{3n} - \tan(\ln 4)\right) + \frac{5 \cdot \cos\left(\frac{n+m}{20n}\right)}{e^{nm}} \quad (1)$$

Листинг 1: Задание 2.1 – математическое выражение (task2_1.lisp)

```
1 (defun task-2-1 (m n)
2   "Вычисляет математическое выражение для варианта 1."
3   (+ (sin (- (/ (+ (* m n) 2)
4                (* 3 n))
5              (tan (log 4))))
6       (/ (* 5 (cos (/ (+ n m)
7                       (* 20 n))))
8          (exp (* n m)))))
9
10 ;; Тестовый запуск
11 (format t "Результат для m=2, n=3: ~A~%"
12         (task-2-1 2 3))
13 (format t "Результат для m=1, n=1: ~A~%"
14         (task-2-1 1 1))
```

3.2 Задание 2.2 – функция $Y(x)$ с условным предложением **COND**

Вариант 1. Реализовать функцию:

$$Y(x) = \begin{cases} \sin(x/a) + \cos(ax), & x \geq 3, \\ \ln(x), & 1 < x < 2. \end{cases} \quad (2)$$

Константа a задаётся как аргумент функции.

Листинг 2: Задание 2.2 – функция $Y(x)$ (task2_2.lisp)

```
1 (defun y-func (x a)
2   "Вычисляет Y(x) по условиям варианта 1."
3   (cond
4     ((>= x 3)
5      (+ (sin (/ x a)) (cos (* a x))))
6     ((and (> x 1) (< x 2))
7      (log x))
8     (t
9      (format nil
10              "x=~A не входит в область определения" x))))
11
12 (defparameter a 2.0)
13
14 (format t "Y(4, a=~A) = ~A~%" a (y-func 4 a))
15 (format t "Y(1.5, a=~A) = ~A~%" a (y-func 1.5 a))
16 (format t "Y(2.5, a=~A) = ~A~%" a (y-func 2.5 a))
```

В данной реализации используется условное предложение **COND**, которое последовательно проверяет условия и выполняет соответствующую ветвь. Условие **t** в конце является ветвью по умолчанию.

3.3 Задание 2.3 – рекурсивная функция вычисления факториала

Вариант 1. Вычислить факториал заданного числа.

Тип рекурсии: линейная (хвостовая). Хвостовая рекурсия характеризуется тем, что рекурсивный вызов является последней операцией в функции. Для реализации хвостовой рекурсии используется вспомогательная функция с аккумулятором, который накапливает промежуточный результат. Это позволяет компилятору оптимизировать рекурсию до цикла (TCO – Tail Call Optimization).

Листинг 3: Задание 2.3 – факториал (task2_3.lisp)

```
1 (defun factorial (n)
2   "Вычисляет факториал числа n.
3   Тип рекурсии: хвостовая."
4   (if (< n 0)
5       (error "Факториал не определен для отриц. чисел")
6       (factorial-helper n 1)))
7
8 (defun factorial-helper (n acc)
9   "Вспомогательная функция с аккумулятором."
10  (if (<= n 1)
11      acc
12      (factorial-helper (- n 1) (* acc n))))
13
14 ;; Тестовые запуски
15 (format t "0! = ~A~%" (factorial 0))
16 (format t "1! = ~A~%" (factorial 1))
17 (format t "5! = ~A~%" (factorial 5))
18 (format t "10! = ~A~%" (factorial 10))
19 (format t "20! = ~A~%" (factorial 20))
```


3.4 Задание 3 – обработка текста с использованием функционалов

Вариант 1. Сделать заглавной первую букву первого слова каждого предложения.

Реализация построена на функциональном подходе:

- текст разбивается на список предложений (по точке);
- каждое предложение обрабатывается функцией `process-sentence`;
- для применения функции к каждому элементу списка используется функционал `mapcar`;
- для объединения результатов – функционал `reduce`;
- циклы `for/while` не используются.

Листинг 4: Задание 3 – обработка текста (task3_text.lisp)

```
1 (defun capitalize-first-char (str)
2   "Делает первый буквенный символ строки заглавным."
3   (if (or (null str) (= (length str) 0))
4       str
5       (let ((trimmed (string-trim
6                       '(#\Space #\Tab #\Newline) str)))
7         (if (= (length trimmed) 0)
8             str
9             (concatenate 'string
10                          (string (char-upcase (char trimmed 0)))
11                          (subseq trimmed 1))))))
12
13 (defun process-sentence (sentence)
14   "Обрабатывает одно предложение."
15   (capitalize-first-char
16     (string-trim '(#\Space #\Tab #\Newline) sentence)))
17
18 (defun process-text (text)
19   "Главная функция: обрабатывает весь текст."
20   (let* ((sentences (split-into-sentences text))
21          (processed (mapcar #'process-sentence sentences)))
22     (concatenate 'string
23                  (join-sentences processed) ".")))
24
25 ;; Тест
26 (defvar *input-text*
27   "hello world. this is a test. functional programming is great
28   . lisp is powerful")
29 (format t "Результат:~%~A~%"
30         (process-text *input-text*))
```

4 РЕАЛИЗАЦИЯ ЗАДАНИЙ НА ИМПЕРАТИВНОМ ЯЗЫКЕ PYTHON

Для сравнения функционального и императивного подходов те же задания были реализованы на языке Python. В Python-версии используются обычные переменные, условные операторы `if/elif/else` и циклы, что соответствует императивному стилю программирования.

4.1 Задание 2.1 – вычисление математического выражения

Листинг 5: Задание 2.1 на Python (task2_1.py)

```
1 import math
2
3 def task_2_1(m, n):
4     first_part = (m * n + 2) / (3 * n) - math.tan(math.log(4))
5     second_part = 5 * math.cos((n + m) / (20 * n)) / math.exp(n
6         * m)
7     return math.sin(first_part) + second_part
8
9 print(f"Результат task-2-1 для m=2, n=3: {task_2_1(2, 3)}")
10 print(f"Результат task-2-1 для m=1, n=1: {task_2_1(1, 1)}")
```

4.2 Задание 2.2 – функция $Y(x)$

Листинг 6: Задание 2.2 на Python (task2_2.py)

```
1 import math
2
3 def y_func(x, a):
4     if x >= 3:
5         return math.sin(x / a) + math.cos(a * x)
6     elif 1 < x < 2:
7         return math.log(x)
8     else:
9         return f"Значение x={x} не входит в область определения"
10
11 a = 2.0
12
13 print(f"Y(4, a={a}) = {y_func(4, a)}")
14 print(f"Y(1.5, a={a}) = {y_func(1.5, a)}")
15 print(f"Y(2.5, a={a}) = {y_func(2.5, a)}")
```

4.3 Задание 2.3 – вычисление факториала

Листинг 7: Задание 2.3 на Python (task2_3.py)

```
1 def factorial(n):
2     if n < 0:
3         raise ValueError("Факториал не определен для отрицательных чисел")
4
5     result = 1
6     for value in range(2, n + 1):
7         result *= value
8
9     return result
10
11 for number in (0, 1, 5, 10, 20):
12     print(f"{number}! = {factorial(number)}")
```

4.4 Задание 3 – обработка текста

Листинг 8: Задание 3 на Python (task3_text.py)

```
1 def process_text(text):
2     raw_sentences = text.split(".")
3     processed_sentences = []
4
5     for sentence in raw_sentences:
6         trimmed = sentence.strip()
7         if not trimmed:
8             continue
9
10        processed = trimmed[0].upper() + trimmed[1:]
11        processed_sentences.append(processed)
12
13    return ". ".join(processed_sentences) + "."
14
15 input_text = (
16     "hello world. this is a test. functional programming is
17     great. "
18     "lisp is powerful"
19 )
20 print("Исходный текст:")
21 print(input_text)
22 print()
23 print("Результат:")
24 print(process_text(input_text))
```

5 ОСНОВНЫЕ КОНСТРУКЦИИ ЯЗЫКА

В ходе выполнения заданий были использованы следующие конструкции Common Lisp.

Функция `defun`. Определение именованных функций. Синтаксис: `(defun имя (параметры) тело)`.

Условное предложение `cond`. Аналог конструкции `switch/case`. Последовательно проверяет условия и выполняет тело первого истинного. Условие `t` является ветвью по умолчанию.

Условный оператор `if`. Простое ветвление: `(if условие then-ветвь else-ветвь)`.

Конструкция `let/let*`. Локальные привязки переменных. `let*` допускает использование ранее определённых переменных в последующих привязках.

Функционал `mapcar`. Функция высшего порядка, применяющая заданную функцию к каждому элементу списка и возвращающая список результатов.

Функционал `reduce`. Сворачивает список в одно значение, последовательно применяя бинарную функцию к аккумулятору и очередному элементу.

Рекурсия. В Lisp рекурсия является основным средством организации повторяющихся вычислений. В задании 2.3 использована хвостовая рекурсия с аккумулятором.

Математические функции. Встроенные функции `sin`, `cos`, `tan`, `log`, `exp`, `sqrt`, `abs` для вычисления математических выражений.

Строковые функции. `string-trim`, `subseq`, `concatenate`, `char-upcase` – для обработки текстовых данных.

6 СРАВНЕНИЕ ФУНКЦИОНАЛЬНОГО И ИМПЕРАТИВНОГО ПОДХОДОВ

Реализации на Common Lisp и Python дают одинаковые результаты на выбранных тестовых данных, однако отличаются способом описания вычислений.

Таблица 1: Сравнение реализаций на Common Lisp и Python

Критерий	Common Lisp	Python
Математические выражения	Записываются в префиксной форме: операция указывается перед аргументами.	Записываются в привычной инфиксной форме, близкой к математической записи.
Условные конструкции	Используется <code>cond</code> , где каждая ветвь содержит условие и выражение-результат.	Используются операторы <code>if</code> , <code>elif</code> , <code>else</code> , которые выполняют последовательные команды.
Повторение действий	В задании факториал реализован через хвостовую рекурсию с аккумулятором.	Факториал реализован через цикл <code>for</code> и изменяемую переменную <code>result</code> .
Обработка текста	Список предложений преобразуется функциями <code>mapcar</code> и <code>reduce</code> .	Предложения обрабатываются последовательно в цикле, результат накапливается в списке.
Стиль программы	Акцент сделан на вычислении значений выражений и применении функций.	Акцент сделан на последовательности действий и изменении промежуточного состояния.

Наиболее заметное отличие проявляется в задании 2.3. В Common Lisp факториал описывается рекурсивно: функция передает новое значение аккумулятора в следующий вызов. В Python та же задача естественно выражается циклом, где переменная `result` изменяется на каждой итерации.

В задании 3 функциональная реализация показывает применение функций высшего порядка: одна функция применяется ко всем предложениям, а затем список результатов сворачивается в строку. Императивная Python-версия явно проходит по предложениям, проверяет каждую строку и добавляет обработанный результат в список. Такой вариант проще воспринимается как последовательный алгоритм, но сильнее зависит от изменяемого состояния.

7 РЕЗУЛЬТАТЫ РАБОТЫ КОДА

В данном разделе приведены экранные формы, отображающие результаты выполнения программ в среде SBCL (запуск через `sbcl -script`) и Python (запуск через `python3`).

```
Задание 2.1 - sbcl --script task2_1.lisp  
  
Результат task-2-1 для m=2, n=3: 0.98299  
Результат task-2-1 для m=1, n=1: 2.7681901
```

Рис. 1: Результат выполнения задания 2.1

```
Задание 2.2 - sbcl --script task2_2.lisp  
  
Y(4, a=2.0) = 0.7637974  
Y(1.5, a=2.0) = 0.4054651  
Y(2.5, a=2.0) = Значение x=2.5 не входит в область определения
```

Рис. 2: Результат выполнения задания 2.2

```
Задание 2.3 - sbcl --script task2_3.lisp  
  
0! = 1  
1! = 1  
5! = 120  
10! = 3628800  
20! = 2432902008176640000
```

Рис. 3: Результат выполнения задания 2.3

```
Задание 3 - sbcl --script task3_text.lisp  
  
Исходный текст:  
hello world. this is a test. functional programming is great. lisp is powerful  
  
Результат:  
Hello world. This is a test. Functional programming is great. Lisp is powerful.
```

Рис. 4: Результат выполнения задания 3 (обработка текста)

```
Задание 2.1 – python3 task2_1.py

Результат task-2-1 для m=2, n=3: 0.9829899917338274
Результат task-2-1 для m=1, n=1: 2.76818998111102477
```

Рис. 5: Результат выполнения задания 2.1 на Python

```
Задание 2.2 – python3 task2_2.py

Y(4, a=2.0) = 0.7637973930170682
Y(1.5, a=2.0) = 0.4054651081081644
Y(2.5, a=2.0) = Значение x=2.5 не входит в область определения
```

Рис. 6: Результат выполнения задания 2.2 на Python

```
Задание 2.3 – python3 task2_3.py

0! = 1
1! = 1
5! = 120
10! = 3628800
20! = 2432902008176640000
```

Рис. 7: Результат выполнения задания 2.3 на Python

```
Задание 3 – python3 task3_text.py

Исходный текст:
hello world. this is a test. functional programming is great. lisp is powerful

Результат:
Hello world. This is a test. Functional programming is great. Lisp is powerful.
```

Рис. 8: Результат выполнения задания 3 на Python

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были получены следующие результаты:

1 Установлен и настроен интерпретатор SBCL (Steel Bank Common Lisp) и среда разработки Visual Studio Code с расширением Alive для интерактивной работы с Common Lisp.

2 Реализовано вычисление математического выражения варианта 1 с использованием встроенных математических функций Lisp.

3 Реализована функция $Y(x)$ с условным предложением `COND`, корректно обрабатывающая различные диапазоны входных значений.

4 Реализована рекурсивная функция вычисления факториала с использованием хвостовой рекурсии и аккумулятора.

5 Разработана программа обработки текста на естественном языке с применением функционалов `mapcar` и `reduce`, без использования циклов и изменяемых переменных.

6 Выполнена дополнительная реализация заданий на императивном языке Python с использованием условных операторов, циклов и изменяемых переменных.

7 Проведено сравнение функционального и императивного подходов. Установлено, что оба варианта дают одинаковые результаты на тестовых данных, но отличаются способом организации вычислений: Common Lisp ориентирован на применение функций и рекурсию, а Python – на последовательность команд и явное изменение состояния.

8 Изучены основные конструкции языка Common Lisp: определение функций, условные выражения, рекурсия, функции высшего порядка, работа со строками и списками.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 HomeLisp – учебная реализация Лиспа [Электронный ресурс]. – Режим доступа: <https://homelisp.ru/>. – Дата доступа: 10.04.2026.
- 2 Haskell home page [Электронный ресурс]. – Режим доступа: <https://haskell.org>. – Дата доступа: 10.04.2026.
- 3 F# Software Foundation [Электронный ресурс]. – Режим доступа: <https://fsharp.org>. – Дата доступа: 10.04.2026.
- 4 Официальный сайт Clojure [Электронный ресурс]. – Режим доступа: <https://clojure.org>. – Дата доступа: 10.04.2026.
- 5 Официальный сайт Scala [Электронный ресурс]. – Режим доступа: <https://scala-lang.org>. – Дата доступа: 10.04.2026.
- 6 Steel Bank Common Lisp – SBCL [Электронный ресурс]. – Режим доступа: <http://www.sbcl.org/>. – Дата доступа: 02.06.2026.
- 7 Alive: Common Lisp extension for VS Code [Электронный ресурс]. – Режим доступа: <https://marketplace.visualstudio.com/items?itemName=rheller.alive>. – Дата доступа: 02.06.2026.
- 8 Visual Studio Code [Электронный ресурс]. – Режим доступа: <https://code.visualstudio.com/>. – Дата доступа: 02.06.2026.
- 9 Мерц, Д. Функциональное программирование на Python / Д. Мерц. – М. : АСТ, 2026. – 128 с.
- 10 Python 3 Documentation [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/>. – Дата доступа: 03.06.2026.